FastAPI Book Management Lab Report

Arielle Whittle
Clark Atlanta University

Advanced Programming
November 8, 2026

Project overview

For this mini-lab, I ran a FastAPI server and used a Python command-line client to manage a collection of books. I tested creating, reading, updating, and deleting books, along with all five additional features. The screenshots show successful requests, changes to the collection, and the expected error responses.

Setup and implementation

I opened the fastapi_lab folder in VS Code on my Mac, created and activated a Python virtual environment, and installed FastAPI, Uvicorn, and requests from requirements.txt. I started the server with python server/main.py and used a separate terminal for the client commands. I also tested an endpoint through the Swagger UI at /docs.

The server code is stored in server/main.py, and the command-line client is stored in client/client.py. The server keeps the books in a Python list. Each book has an ID, title, author, year, genre, and ratings. Changes remain available while the server is running, but restarting the server resets the collection.

JSON serialization

The Author model is nested inside the Book model, so the API exchanges more than simple text values. Pydantic validates incoming data and converts the models into JSON responses. The client sends Python dictionaries using the requests library's json argument and reads responses using response.json(). Ratings are represented as an array of integers, and an optional author email can appear as null.

Additional features completed

- Genre search returns matching books without requiring the same capitalization.
- Ratings are accepted during creation and full updates.
- PATCH changes selected fields while preserving the other values.
- HTTP Basic authentication checks the lab username and password.
- The count endpoint reports the current number of books.

Testing evidence

1 Project setup and server startup

I created a virtual environment, installed the dependencies, and started server/main.py. The terminal shows Application startup complete and Uvicorn running at http://127.0.0.1:8000. The server was restarted successfully after an earlier shutdown.

```
[notice] A new release of pip is available: 23.2.1 -> 26.3
[notice] To update, run: pip install --upgrade pip
INFO:     Started server process [37104]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
^CINFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [37104]
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python server/main.py
INFO:     Started server process [37189]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Figure 1. Project setup and server startup.

2 Listing the initial books

The list command returned HTTP 200 and both initial books, The Great Gatsby and 1984. Each book includes a nested author object and a ratings list. George Orwell has a null email, showing that the optional email field is supported.

```
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % source .venv/bin/activate
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python client/client.py list
Status Code: 200
[
  {
    "id": 1,
    "title": "The Great Gatsby",
    "author": {
      "name": "F. Scott Fitzgerald",
      "email": "fscott@example.com"
    },
    "year": 1925,
    "genre": "Fiction",
    "ratings": []
  },
  {
    "id": 2,
    "title": "1984",
    "author": {
      "name": "George Orwell",
      "email": null
    },
    "year": 1949,
    "genre": "Dystopian",
    "ratings": []
  }
]
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab %
```

Figure 2. Listing the initial books.

Testing evidence continued

3 Retrieving a book by ID

The get 1 command returned HTTP 200 and The Great Gatsby. The response includes its ID, title, author, publication year, genre, and empty ratings list.

```
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python client/client.py get 1
Status Code: 200
{
  "id": 1,
  "title": "The Great Gatsby",
  "author": {
    "name": "F. Scott Fitzgerald",
    "email": "fscott@example.com"
  },
  "year": 1925,
  "genre": "Fiction",
  "ratings": []
}
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab %
```

Figure 3. Retrieving a book by ID.

4 Confirming the retrieval result

This additional view shows the same successful list and get operations. The single-book response preserves the nested author structure used in the collection response.

```
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python client/client.py get 1
Status Code: 200
{
  "id": 1,
  "title": "The Great Gatsby",
  "author": {
    "name": "F. Scott Fitzgerald",
    "email": "fscott@example.com"
  },
  "year": 1925,
  "genre": "Fiction",
  "ratings": []
}
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab %
```

Figure 4. Confirming the retrieval result.

Testing evidence continued

5 Creating a book with ratings

The create command added Pride and Prejudice as book 3 and returned HTTP 201 Created. The JSON response contains Jane Austen's name and email, the year 1813, genre Romance, and ratings of 4, 5, and 3.

```
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python client/client.py create "Pride and Prejudic
e" "Jane Austen" "jane@example.com" 1813 "Romance" --ratings 4,5,3
Status Code: 201
{
  "id": 3,
  "title": "Pride and Prejudice",
  "author": {
    "name": "Jane Austen",
    "email": "jane@example.com"
  },
  "year": 1813,
  "genre": "Romance",
  "ratings": [
    4,
    5,
    3
  ]
}
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab %
```

Figure 5. Creating a book with ratings.

6 Updating a complete book

The update command used PUT to replace book 3 and returned HTTP 200. The genre changed to Classic and the ratings changed to 5 and 5. The response also includes the title and nested author data.

```
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python client/client.py update 3 "Pride and Prejud
ice" "Jane Austen" "jane@example.com" 1813 "Classic" --ratings 5,5
Status Code: 200
{
  "id": 3,
  "title": "Pride and Prejudice",
  "author": {
    "name": "Jane Austen",
    "email": "jane@example.com"
  },
  "year": 1813,
  "genre": "Classic",
  "ratings": [
    5,
    5
  ]
}
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab %
```

Figure 6. Updating a complete book.

Testing evidence continued

7 Searching books by genre

The search-genre fiction command returned HTTP 200 and The Great Gatsby. Although the command used lowercase fiction, the stored genre is Fiction, confirming that the genre search is case-insensitive.

```
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python client/client.py search-genre fiction
Status Code: 200
[
  {
    "id": 1,
    "title": "The Great Gatsby",
    "author": {
      "name": "F. Scott Fitzgerald",
      "email": "fscott@example.com"
    },
    "year": 1925,
    "genre": "Fiction",
    "ratings": []
  }
]
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab %
```

Figure 7. Searching books by genre.

8 Updating only the title with PATCH

The patch command changed book 3's title to Pride and Prejudice (Updated) and returned HTTP 200. The author, year 1813, genre Classic, and ratings of 5 and 5 remained unchanged.

```
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab % python client/client.py patch 3 --title "Pride and Prejudice
(Updated)"
Status Code: 200
{
  "id": 3,
  "title": "Pride and Prejudice (Updated)",
  "author": {
    "name": "Jane Austen",
    "email": "jane@example.com"
  },
  "year": 1813,
  "genre": "Classic",
  "ratings": [
    5,
    5
  ]
}
(.venv) ariellewhittle@Arielles-MacBook-Air fastapi_lab %
```

Figure 8. Updating only the title with PATCH.

Testing evidence continued

9 Counting the books

The count command returned HTTP 200 with a count of 3. This matches the two initial books plus the newly created book.

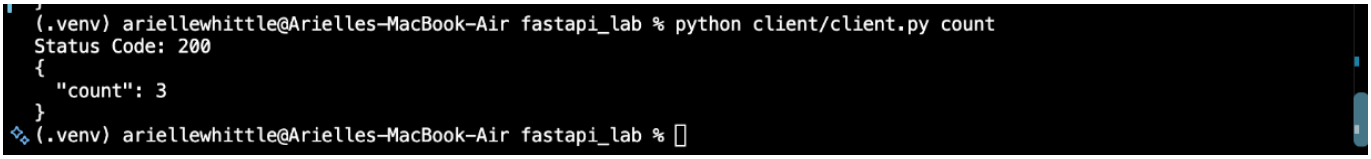


Figure 9. Counting the books.

10 Deleting a book and checking the count

The delete 3 command returned HTTP 200 and the message Book deleted. Running count afterward returned 2, confirming that the book was removed from the in-memory collection.

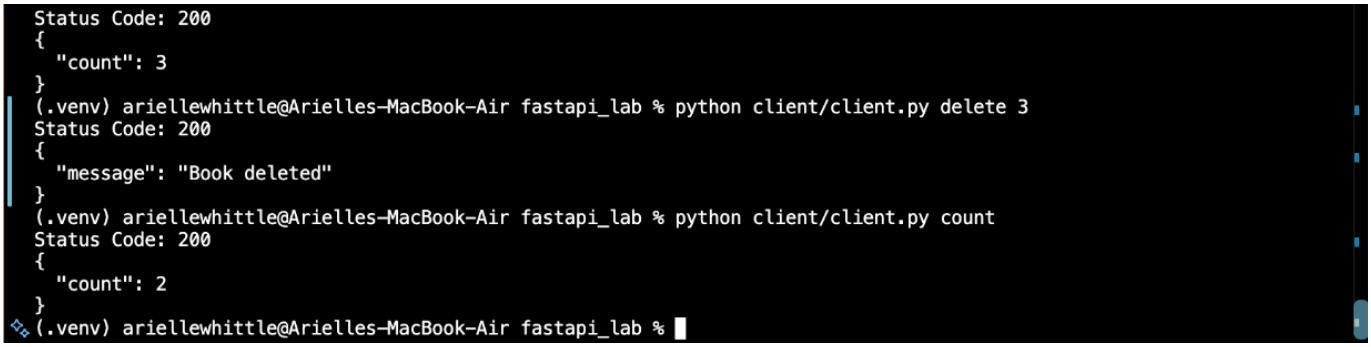


Figure 10. Deleting a book and checking the count.

Testing evidence continued

11 Handling a book that does not exist

The get 999 command returned HTTP 404 with the message Book not found. The client displayed the error response clearly instead of ending with an unhandled exception.

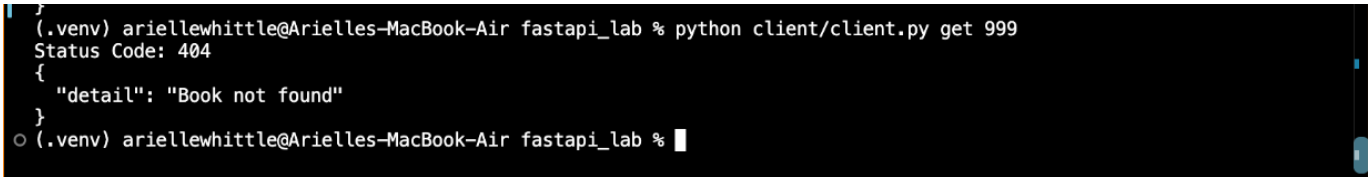


Figure 11. Handling a book that does not exist.

12 Rejecting incorrect credentials

Using --password wrong with the list command returned HTTP 401 and Invalid credentials. This confirms that the server rejects an incorrect password and that the client displays the authentication error.

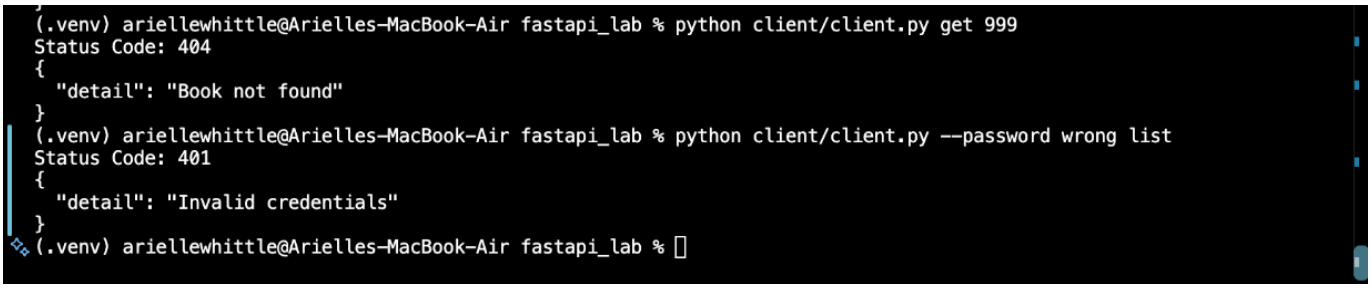


Figure 12. Rejecting incorrect credentials.

Testing evidence continued

13 Testing the API in Swagger UI

The Swagger UI request for book 1 returned HTTP 200 and The Great Gatsby. The response body contains the nested author object, and the response header identifies the content type as application/json.



Figure 13. Testing the API in Swagger UI.

Conclusion

The results show that the client and server communicated successfully using HTTP and JSON. Creating a book returned 201, and the successful read, update, PATCH, delete, search, and count requests returned 200. Deleting book 3 reduced the count from 3 to 2. The 404 and 401 responses showed that missing records and incorrect credentials were handled as expected. This lab helped me connect command-line inputs, REST endpoints, and nested JSON data in one working application.